Guía definitiva

Lakehouse en AWS con Terraform, Glue, Iceberg y Athena

10M+ de filas · menos de 5 USD · todo como código · Spark nativo (sin spark.sql)

Bases de Datos II · Equipo de 2 personas · Sustentación: viernes 13 de noviembre de 2026

Qué cambió en esta versión. (1) Los jobs de Spark usan solo la API de DataFrames: no hay `spark.sql`, ni `MERGE INTO`, ni `expr()`. (2) Todo el código está en inglés (nombres, comentarios y mensajes); la explicación sigue en español. (3) Los jobs viven en `jobs/` y la lógica reutilizable en la nueva carpeta `helpers/`. (4) La sección de Terraform no se modificó; solo se añadió el Anexo A (opcional) para que Glue reciba la carpeta `helpers/`. (5) El código completo de los jobs y helpers ahora está en esta guía (sección 6).

Cómo usar esta guía. Es el plan de ejecución: decisiones, pasos, comandos, SQL, evidencias y errores típicos, más el código de los jobs y los helpers con su explicación. El Terraform completo y el generador de datos siguen en el documento editable compartido; de ahí se copian al repositorio.

1. Resumen y decisiones de diseño

Plataforma lakehouse en AWS (us-east-1) creada solo con Terraform: landing → bronze → silver (Iceberg) → gold (estrella en Iceberg), orquestada con Step Functions y consultada con Athena. La sustentación se hace sobre más de 10M de filas en fact_sales, con alerta de AWS Budgets en 5 USD.

Tema	Decisión
Tablas Iceberg	12 tablas: 5 en silver (customers, products, stores, orders, order_items) y 7 en gold (fact_sales, 5 dimensiones y el mart agg_sales_dept_age_quarter). Los logs web_events se quedan en landing y bronze.
Volumen	4,3M pedidos ≈ 10,6M líneas en fact_sales (media de 2,48 ítems por pedido), con margen sobre 10M tras los rechazos.
Bucket	1 bucket con prefijos: landing/ silver/ gold/ scripts/ athena-results/ glue-temp/ warehouse/.
Lotes	El lote 2 reenvía también clientes, productos y tiendas (snapshot diario), para que el job nunca lea particiones vacías.
Surrogate keys	abs(xxhash64(clave_natural)): deterministas e idempotentes. Fila -1 / Unknown en cada dimensión.
Privacidad	El PII (nombre, correo) vive solo en silver. Gold no lo contiene.
Calidad	Los inválidos van a silver/_rejects/; el job falla si supera el 5 % de rechazos.
Particionado	months(order_date) en orders y fact_sales. Parquet con compresión zstd.
Spark nativo	Solo API de DataFrames y DataFrameWriterV2. Cero spark.sql / MERGE INTO / expr(). El upsert se implementa con left_anti join + overwrite (sección 6.4).
Código en inglés	Variables, funciones, columnas, comentarios y mensajes de log en inglés. La documentación va en español.
Carpetas de código	jobs/ (los 2 scripts que ejecuta Glue) y helpers/ (funciones reutilizables importadas por los jobs).
Usuario IAM	Un solo usuario compartido lakehouse-admin con AdministratorAccess, MFA y un único CSV de access key, usado por ambos con AWS CLI v2.
Roles de servicio	Tres roles de mínimo privilegio (crawlers, jobs de Spark, Step Functions), creados por Terraform.

Arquitectura medallón: qué hace cada capa

Capa	Pregunta que responde	Qué contiene
Landing	¿Qué llegó, exactamente como llegó?	CSV.gz y JSON.gz en landing/<fuente>/<tabla>/ingest_date=AAAA-MM-DD/. Nada se modifica.
Bronze	¿Qué estructura tiene?	Tablas raw_* creadas por los crawlers sobre los mismos archivos. Duplicados y datos sucios siguen ahí a propósito.
Silver	¿Qué datos son confiables?	Una tabla Iceberg por entidad del OLTP: tipificada, normalizada, validada y deduplicada. Se escribe con upsert (idempotente).
Gold	¿Qué necesita el negocio?	Estrella: fact_sales + dim_customer, dim_product, dim_store, dim_geography, dim_date + mart agregado.

Por qué 5 tablas en silver: una por entidad transaccional, que son las que necesita gold. Así el upsert toca solo lo que cambia y el borrado de un cliente se hace por su clave. **Por qué 7 en gold:** la rúbrica pide fact_sales más 5 dimensiones; el mart es un extra que precalcula la pregunta del gerente y la responde sin joins.

Diccionario de fact_sales

Columna	Tipo	Nota
order_item_id	bigint	Clave natural (grano)
order_id	bigint	Dimensión degenerada

Columna	Tipo	Nota
date_key	int	FK a dim_date (yyyyMMdd)
customer_key, product_key, store_key, geography_key	bigint	FK; -1 si es desconocido
order_date	date	Columna de partición
quantity, discount_pct	int, double	
unit_price, gross_amount, discount_amount, net_amount	decimal(14,2)	net = gross - discount
order_status, payment_method	string	
_loaded_at	timestamp	Auditoría

2. Presupuesto (alerta obligatoria en 5 USD)

Precio de referencia de Glue: ≈ 0,44 USD por DPU-hora, facturado por segundo. Verifíquenlo en la página de precios de AWS.

Concepto	Realista (USD)	Conservador (USD)
Mini: ~16 corridas de job con 2 workers	0,6	0,9
Crawlers: 4 corridas	0,4	0,6
Full lote 1: silver + gold con 8 workers y auto-scaling	1,0	1,8
Full lote 2: upsert sobre tablas de 10M	0,4	0,9
Athena, S3, Step Functions, Catalog	0,2	0,3
Total	**≈ 2,6**	**≈ 4,5**

Compuertas (no se negocian)

- Antes del full lote 1: gasto acumulado de Glue ≤ 1,5 USD.
- Antes del full lote 2: acumulado ≤ 3,0 USD. Si no, el upsert se demuestra con las evidencias del mini.
- Tope duro por corrida: timeout de 25 min × 8 workers ≈ 1,47 USD como máximo.
- Nunca dos corridas full simultáneas ni un job en Running olvidado.
- Antes de subir un script: `python -m py_compile jobs/*.py helpers/*.py`.
- Budgets avisa con retraso y cuenta toda la cuenta; el freno real es el comando de gasto de abajo.

```
for JOB in lakehouse-bronze-to-silver lakehouse-silver-to-gold; do
  aws glue get-job-runs --job-name $JOB --query "JobRuns[].DPUSeconds" --output text
done | tr '\t' '\n' | awk '{s+=$1} END {printf "Glue total so far: %.2f USD\n", s/3600*0.44}'
```

Cost Explorer tiene ~24 h de retraso, por eso el gasto real se calcula con las DPU-segundos de los jobs.

3. Cuenta AWS e IAM

3.1 Qué se hace a mano (una vez) y qué va por Terraform

A mano	Por Terraform
MFA del root; activar acceso de IAM a Billing; Cost Explorer; alias de cuenta	Bucket de S3 y sus prefijos
Crear el usuario lakehouse-admin, su MFA y su access key	Bases de Glue, crawlers y jobs
aws configure --profile lakehouse en cada computador	Workgroup de Athena, Budgets, Step Functions
	Los tres roles de IAM y sus permisos; subida de jobs/*.py al bucket

En la consola de AWS solo se mira (capturas, consultas en Athena, iniciar una ejecución de Step Functions, ver logs). No se crea, edita ni borra infraestructura a mano: si no está en Terraform, no cuenta.

3.2 Usuario root (una persona, una vez)

- Consola → menú con su nombre → Security credentials → Assign MFA device (Authenticator app).
- Menú con su nombre → Account → IAM user and role access to Billing information → Edit → Activate IAM Access. Solo el root puede hacerlo; sin esto fallan los Budgets.
- Billing preferences: activar alertas por correo. Cost Explorer → Launch (tarda ~24 h).
- Selector de región: N. Virginia (us-east-1). Verificarlo cada vez.
- IAM → Account Alias → Create: lakehouse-equipo.

3.3 Usuario compartido lakehouse-admin

- IAM → Users → Create user → nombre lakehouse-admin; marcar acceso a consola; contraseña autogenerada.
- Attach policies directly → AdministratorAccess → Create user. Guardar URL de login, usuario y contraseña.
- Entrar como lakehouse-admin → Security credentials → Assign MFA device (ideal: ambos escanean el mismo QR).
- Security credentials → Create access key → caso Command Line Interface (CLI) → Download .csv.
- Compartir el CSV por un canal privado y borrarlo de chats y correos. Nunca va a Git.
- Cada persona: `aws configure --profile lakehouse` con los valores del CSV, región us-east-1, formato json.
- Verificar en ambos: `aws sts get-caller-identity` debe mostrar lakehouse-admin y el mismo número de cuenta.

3.4 Los tres roles (los crea Terraform en iam.tf)

Rol	Quién lo asume	Permisos	Para qué
lakehouse-crawler-role	Glue	AWSGlueServiceRole + s3:GetObject solo en landing/*	Los crawlers leen la landing y crean las tablas raw_*
lakehouse-glue-role	Glue	glue:* solo sobre lakehouse_*; lectura/escritura en el bucket; logs en /aws-glue/*	Los jobs de Spark escriben silver y gold en Iceberg
lakehouse-sfn-role	Step Functions	Solo StartJobRun, GetJobRun, GetJobRuns, BatchStopJobRun sobre los 2 jobs	Orquesta el pipeline

Para la sustentación: el usuario operativo es administrador por requisito del curso y tiene MFA. Lo que corre sin supervisión lo hace con tres roles de mínimo privilegio, definidos como código. Matiz honesto: glue:* está limitado por recurso a lakehouse_*, pero podría reducirse a la lista exacta de acciones.

Nota sobre los jobs: para construir el nombre del bucket, los jobs llaman a `sts:GetCallerIdentity`, que no requiere permisos en IAM.

3.5 Un solo estado de Terraform

- Solo una persona ejecuta terraform apply y destroy; la otra desarrolla jobs/, helpers/, data/ y sql/ y lo sube a Git.
- Avisen en el chat antes de cada apply.
- Si ambos quieren aplicar: bucket de estado remoto con bloqueo (use_lockfile = true, Terraform ≥ 1.10), creado antes del primer apply.

4. Repositorio y Terraform

Esta sección no se modificó: el Terraform ya está funcional. La única adición es la carpeta helpers/ en la estructura y el Anexo A (opcional), que explica qué cambiar en glue.tf para que los jobs puedan importarla.

4.1 Estructura y primeros archivos

```
lakehouse-aws/
├── terraform/
├── jobs/                    # scripts that Glue runs
│   ├── bronze_to_silver.py
│   └── silver_to_gold.py
├── helpers/                 # reusable code imported by the jobs
│   ├── __init__.py
│   ├── config.py
│   ├── spark_session.py
│   ├── cleaning.py
│   ├── quality.py
│   ├── iceberg_io.py
│   └── keys.py
├── data/   ├── sql/   ├── docs/evidence/   └── README.md
```

```
# .gitignore
.terraform/
*.tfstate*
*.tfvars
!terraform.tfvars.example
tfplan
data/out*/
.env
__pycache__/
helpers.zip
```

Placeholders para que terraform apply funcione hoy (desde la raíz del repo). Terraform los necesita porque glue.tf calcula su hash con filemd5:

```
mkdir -p jobs
echo 'print("pending")' > jobs/bronze_to_silver.py
echo 'print("pending")' > jobs/silver_to_gold.py
# PowerShell: mkdir jobs -Force ; Set-Content jobs\bronze_to_silver.py 'print("pending")' ...
```

4.2 Archivos de terraform/

Archivo	Contenido
versions.tf	Proveedor AWS ~> 5.0 y default_tags (Project, Environment, Owner, ManagedBy, Course).
variables.tf	region, project, environment, owner, alert_emails, budget_limit_usd, glue_version, glue_workers, glue_timeout_min.
terraform.tfvars.example	Plantilla con owner y alert_emails. Se copia a terraform.tfvars (ignorado por Git) y se llena con valores reales.
s3.tf	Bucket lakehouse-<cuenta>-us-east-1, bloqueo público, cifrado AES256, ciclo de vida (athena-results 7 d, glue-temp 3 d) y prefijos.
athena.tf	Workgroup lakehouse-wg, engine v3, tope de 5 GB por consulta.
budgets.tf	Presupuesto de 5 USD con alertas al 40 %, 80 %, 100 % real y 100 % pronosticado.
iam.tf	Los tres roles de la sección 3.4.
glue.tf	Bases bronze/silver/gold, clasificador CSV, 2 crawlers con table_prefix = "raw_", subida de ../jobs/*.py y 2 jobs.
stepfunctions.tf	Máquina de estados definida con jsonencode (sin archivo .tpl).
outputs.tf	Nombre del bucket, workgroup, ARN de la máquina de estados y las 3 bases.

El contenido completo de cada archivo está en la sección 3.2 del documento editable.

4.3 Desplegar

```
cd terraform
cp terraform.tfvars.example terraform.tfvars    # edit owner and REAL emails
export AWS_PROFILE=lakehouse                    # PowerShell: $env:AWS_PROFILE = "lakehouse"
```

```
aws sts get-caller-identity                    # must show lakehouse-admin
terraform init
terraform fmt && terraform validate
terraform plan -out=tfplan                     # expected: 31 to add
terraform apply tfplan
```

terraform.tfvars.example va a Git; terraform.tfvars no (tiene datos personales). Cada terminal tiene su propio entorno: en Git Bash se usa export, en PowerShell $env:.

4.4 Errores de Terraform ya resueltos (no repetirlos)

Síntoma	Causa y solución
apply_server_side_encryption_by_default falla en s3.tf	Un bloque de una sola línea solo admite atributos. Escribir rule { ... } y su bloque anidado en varias líneas.
filemd5 ... no puede encontrar la ruta	Falta la carpeta jobs/ con los dos scripts. Crear los placeholders desde la raíz del repo.
No valid credential sources found	No hay perfil activo en esa terminal. Activar AWS_PROFILE=lakehouse (Git Bash: export; PowerShell: $env:).
Step Functions: INVALID_JSON_DESCRIPTION ... code 13	Un salto de línea partió "--ingest_date.$" al copiar. Definir la máquina con jsonencode en stepfunctions.tf.
Guiones perdidos al copiar (awsglue, gluetemp, athenaresults)	Debe ser /aws-glue/*, glue-temp/, athena-results/. Comprobar con grep -n "awsglue|gluetemp|athenaresults" *.tf (no debe imprimir nada).
Alertas de Budgets sin llegar	Quedaron los correos de ejemplo en terraform.tfvars. Poner direcciones reales.

4.5 Verificación en la consola (capturas 1 a 8 del mapa de evidencias)

Con el apply terminado (Apply complete! Resources: 31 added): S3 con prefijos; Budgets con 4 alertas; Athena con lakehouse-wg; 3 bases de Glue; 2 jobs; lakehouse-pipeline en Step Functions; 3 roles lakehouse-* y el usuario lakehouse-admin con MFA. Con esto y el diagrama queda el Avance 1.

5. Datos: generador, landing y bronze

El generador (data/generate_data.py, código en la sección 4.1 del documento editable) crea CSV.gz del OLTP y JSON.gz de logs, por fuente y fecha, en archivos de ~250.000 filas (un .gz no se divide; varios archivos permiten paralelizar). Incluye suciedad intencional (duplicados, cantidades negativas, correos inválidos, fechas futuras) para que silver la limpie.

Perfil	Clientes	Productos	Tiendas	Pedidos	Líneas (≈)	Eventos
mini	5.000	500	30	50.000	124.000	20.000
full	300.000	5.000	150	4.300.000	10,6M	1.000.000

```
# mini batch 1
python generate_data.py --profile mini --batch 1 --ingest-date 2026-11-07
# mini batch 2 (4,000 new orders + 1,000 updated to 'returned')
python generate_data.py --profile mini --batch 2 --ingest-date 2026-11-09 --batch1-date 2026-11-07
# full batches 1 and 2
python generate_data.py --profile full --batch 1 --ingest-date 2026-11-11 --out out_full
python generate_data.py --profile full --batch 2 --ingest-date 2026-11-12 --batch1-date 2026-11-11 --out out_full
```

Python local: usen la misma versión minor los dos (3.12 o 3.13 son las más seguras) en un entorno virtual. Los jobs de Glue usan la versión que fija Glue (3.11 con Glue 5.0), no la local.

Subir la landing y catalogar (mini)

```
cd data && python generate_data.py --profile mini --batch 1 --ingest-date 2026-11-07
cd ../terraform
BUCKET=$(terraform output -raw bucket_name)
aws s3 sync ../data/out/landing s3://$BUCKET/landing/
aws glue start-crawler --name lakehouse-bronze-oltp
aws glue start-crawler --name lakehouse-bronze-logs
aws glue get-crawler --name lakehouse-bronze-oltp --query Crawler.State    # repeat until READY

-- Athena, workgroup lakehouse-wg
SHOW TABLES IN lakehouse_bronze;                          -- raw_customers ... raw_web_events
SELECT count(*) FROM lakehouse_bronze.raw_orders;
SELECT * FROM lakehouse_bronze.raw_web_events LIMIT 5;
DESCRIBE lakehouse_bronze.raw_orders;                     -- note the type of ingest_date
```

Evidencias 9 a 11: landing en S3, crawlers en Succeeded, tablas raw_* y conteo en Athena. Los crawlers corren una vez por fuente; para lotes nuevos se usa MSCK REPAIR TABLE en Athena (gratis).

Esquema bronze que asumen los jobs

Importante. El generador de datos no está en este PDF, así que los jobs asumen los nombres de columna de la tabla de abajo. Antes de la primera corrida compárenlos con `DESCRIBE lakehouse_bronze.raw_*` en Athena. Si una columna se llama distinto, se corrige en un solo lugar: la función `prepare_*` de esa entidad en jobs/bronze_to_silver.py.

Tabla bronze	Columnas esperadas (además de ingest_date)
raw_customers	customer_id, first_name, last_name, email, birth_date, updated_at
raw_products	product_id, product_name, category, updated_at
raw_stores	store_id, store_name, city, department, updated_at
raw_orders	order_id, customer_id, store_id, order_date, status, payment_method, updated_at
raw_order_items	order_item_id, order_id, product_id, quantity, unit_price, discount_pct

6. Jobs de Spark en Glue (Spark nativo)

6.1 Teoría: por qué “Spark nativo”

Spark ofrece dos formas de hacer lo mismo: escribir SQL dentro de un string (`spark.sql("SELECT ...")`) o encadenar operaciones con la **API de DataFrames** (`select, filter, join, groupBy, withColumn`). En ambas, Spark construye el mismo plan lógico y lo optimiza igual (Catalyst). Con DataFrames, los errores de nombre aparecen antes, el código se puede dividir en funciones pequeñas y reutilizables (helpers), y cada paso se explica solo. Por eso esta guía usa únicamente: `pyspark.sql.functions`, `Window`, `DataFrame` y `DataFrameWriterV2` (`df.writeTo(...)`).

Antes (SQL en strings)	Ahora (API nativa)
spark.sql("SELECT ... FROM t WHERE ...")	spark.table("t").filter(...).select(...)
MERGE INTO t USING s ON ...	upsert(): left_anti join + unionByName + overwrite (6.4)
CREATE TABLE ... USING iceberg	df.writeTo(t).using("iceberg").createOrReplace()
INSERT OVERWRITE	df.writeTo(t).overwritePartitions()
expr("a + b"), selectExpr	F.col("a") + F.col("b")

Revisión realizada: se buscó en jobs/ y helpers/ cualquier uso de `spark.sql(`, `.sql(`, `expr(` y `selectExpr`. Resultado: ninguno. Las únicas cadenas que contienen “sql” son los imports de `pyspark.sql` y las claves de configuración `spark.sql.catalog.*` del catálogo Iceberg, que son configuración, no consultas. Los SQL de las secciones 7 y 8 se ejecutan en **Athena**, no en Spark.

Decisión sobre MERGE INTO. En PySpark, Iceberg no ofrece una API de DataFrames para MERGE (a diferencia de Delta Lake): solo existe como sentencia SQL. Para cumplir “todo con Spark nativo” se reemplaza por `upsert()`, que logra el mismo resultado (insertar claves nuevas, reemplazar las existentes) y genera snapshots de Iceberg, así que el time travel funciona igual. La evidencia del lote 2 sigue siendo +4.000 pedidos y +1.000 en returned. Si el profesor exige literalmente la sentencia MERGE INTO, habría que usar spark.sql solo para eso: es decisión de ustedes, y conviene confirmarlo antes del viernes.

6.2 Flujo de cada job

Job	Pasos
bronze_to_silver	Por cada una de las 5 entidades: leer la partición ingest_date de bronze → tipificar y normalizar → quality_gate (rechazos a silver/_rejects/, falla si > 5 %) → deduplicar (el registro más reciente por clave) → upsert en Iceberg.
silver_to_gold	Reconstruir las 5 dimensiones (con fila -1 / Unknown y sin PII) → fact_sales solo con los pedidos tocados en el lote → upsert por order_item_id → reconstruir el mart agg_sales_dept_age_quarter desde toda la tabla de hechos.

Por qué validar antes de deduplicar: si se dedupe primero, todas las filas con clave nula caen en el mismo grupo y se reducen a una sola, lo que escondería errores. Validando primero, cada fila inválida se cuenta y se guarda en _rejects/.

6.3 La carpeta helpers/ (código completo y explicación)

Cada archivo tiene una sola responsabilidad. Los jobs quedan cortos y solo orquestan.

helpers/config.py — constantes compartidas

```
"""Shared constants. Change names here, never inside the jobs."""
import boto3

REGION = "us-east-1"
CATALOG = "glue_catalog"          # Iceberg catalog name inside Spark

BRONZE_DB = "lakehouse_bronze"
SILVER_DB = "lakehouse_silver"
GOLD_DB = "lakehouse_gold"

MAX_REJECT_RATE = 0.05            # the job fails above 5 % rejected rows

DATE_START = "2020-01-01"         # range of dim_date
```

```
DATE_END = "2030-12-31"


def get_bucket():
    """Bucket name built by Terraform: lakehouse-<account>-us-east-1."""
    account = boto3.client("sts", region_name=REGION).get_caller_identity()["Account"]
    return f"lakehouse-{account}-{REGION}"


def table_name(database, table):
    """Full Iceberg name, e.g. glue_catalog.lakehouse_silver.orders."""
    return f"{CATALOG}.{database}.{table}"
```

Aquí viven los nombres de bases, el umbral de rechazo (5 %) y el rango de dim_date. `get_bucket()` reconstruye el nombre del bucket con el número de cuenta, así los jobs no dependen de argumentos extra en Terraform.

helpers/spark_session.py — Spark con catálogo Iceberg

```
"""Creates the Spark session with the Iceberg catalog on top of the Glue Data Catalog."""
from awsglue.context import GlueContext
from pyspark import SparkConf, SparkContext

from helpers.config import CATALOG


def create_session(bucket):
    conf = SparkConf()
    conf.set("spark.sql.extensions",
             "org.apache.iceberg.spark.extensions.IcebergSparkSessionExtensions")
    conf.set(f"spark.sql.catalog.{CATALOG}", "org.apache.iceberg.spark.SparkCatalog")
    conf.set(f"spark.sql.catalog.{CATALOG}.catalog-impl",
             "org.apache.iceberg.aws.glue.GlueCatalog")
    conf.set(f"spark.sql.catalog.{CATALOG}.io-impl", "org.apache.iceberg.aws.s3.S3FileIO")
    conf.set(f"spark.sql.catalog.{CATALOG}.warehouse", f"s3://{bucket}/warehouse/")

    glue_context = GlueContext(SparkContext(conf=conf))
    return glue_context.spark_session, glue_context
```

Iceberg necesita un **catálogo**: aquí se llama glue_catalog y guarda los metadatos en el Glue Data Catalog (por eso Athena ve las mismas tablas). Los datos van a S3. La configuración se hace antes de crear el SparkContext, que es el patrón de la documentación de AWS. El job necesita el parámetro `--datalake-formats iceberg` (ya existe según la sección 13).

helpers/cleaning.py — funciones de limpieza por columna

```
"""Small column-level functions to type, normalize and deduplicate data."""
from pyspark.sql import Window
from pyspark.sql import functions as F

EMAIL_REGEX = r"^[^@\s]+@[^@\s]+\.[^@\s]+$"


def clean_text(column):
    """Trim, collapse repeated spaces and turn empty text into null."""
    cleaned = F.trim(F.regexp_replace(column, r"\s+", " "))
    return F.when(cleaned != "", cleaned)


def title_text(column):
    """clean_text + 'juan perez' / 'JUAN PEREZ' -> 'Juan Perez'."""
    return F.initcap(clean_text(column))


def lower_text(column):
    """clean_text + lower case (emails, status, payment method)."""
    return F.lower(clean_text(column))


def is_valid_email(column):
    """True / False, never null."""
    return F.coalesce(column.rlike(EMAIL_REGEX), F.lit(False))


def deduplicate(df, key, order_column):
    """Keep one row per key: the one with the highest order_column."""
    window = Window.partitionBy(key).orderBy(F.col(order_column).desc_nulls_last())
    return (df.withColumn("_rn", F.row_number().over(window))
              .filter(F.col("_rn") == 1)
              .drop("_rn"))
```

`clean_text` recorta y colapsa espacios; si queda vacío devuelve null (`F.when` sin otherwise). `deduplicate` usa una función de ventana: numera las filas de cada clave de la más reciente a la más antigua y se queda con la 1.

helpers/quality.py — la compuerta de calidad

```
"""Data quality gate: separates valid rows from rejects and fails above the threshold."""
from pyspark.sql import functions as F

from helpers.config import MAX_REJECT_RATE


def quality_gate(df, entity, rules, rejects_path, max_rate=MAX_REJECT_RATE):
    """
    rules: list of (reason, bad_condition). A row is rejected when a condition is true;
    the first matching reason is stored in the _reject_reason column.
    """
    reason = F.coalesce(*[F.when(bad, F.lit(name)) for name, bad in rules])
    flagged = df.withColumn("_reject_reason", reason).persist()

    counts = flagged.agg(
        F.count(F.lit(1)).alias("total"),
        F.count("_reject_reason").alias("rejected"),   # count() skips nulls
    ).first()
    total, rejected = counts["total"], counts["rejected"]
    rate = rejected / total if total else 0.0
    print(f"[QUALITY] entity={entity} total={total} rejected={rejected} rate={rate:.4f}")

    (flagged.filter(F.col("_reject_reason").isNotNull())
            .write.mode("overwrite").parquet(rejects_path))

    if rate > max_rate:
        raise ValueError(f"{entity}: reject rate {rate:.2%} is above {max_rate:.0%}")

    return flagged.filter(F.col("_reject_reason").isNull()).drop("_reject_reason")
```

Cada regla es un par (motivo, condición mala). La primera condición que se cumple marca la fila como rechazada y guarda el motivo. El df se guarda en memoria (`persist`) porque se usa tres veces: contar, escribir los rechazos y devolver los válidos. El log `[QUALITY]` es la evidencia 13.

helpers/iceberg_io.py — leer bronze y escribir Iceberg

```
"""Read bronze and write Iceberg tables using only the DataFrame / DataFrameWriterV2 API."""
from pyspark.sql import functions as F
from pyspark.sql.utils import AnalysisException


def read_bronze(glue_context, database, table, ingest_date):
    """Read one ingest_date partition of a crawler table."""
    try:
        frame = glue_context.create_dynamic_frame.from_catalog(
            database=database, table_name=table,
            push_down_predicate=f"ingest_date == '{ingest_date}'")
        return frame.toDF()
    except Exception:
        # ingest_date was typed as date: no predicate, filter after reading
        frame = glue_context.create_dynamic_frame.from_catalog(
            database=database, table_name=table)
        return frame.toDF().filter(F.col("ingest_date").cast("string") == ingest_date)


def table_exists(spark, table):
    try:
        spark.table(table)
        return True
    except AnalysisException:
        return False


def create_or_replace_table(df, table, location, partition_column=None):
    """Create the Iceberg table (or replace its contents) with the DataFrame schema."""
    writer = (df.writeTo(table).using("iceberg")
                .tableProperty("format-version", "2")
                .tableProperty("write.parquet.compression-codec", "zstd")
                .tableProperty("location", location))
    if partition_column:
        writer = writer.partitionedBy(F.months(partition_column))
    writer.createOrReplace()


def upsert(spark, new_df, table, key, location, partition_column=None):
```

```
    """
    Insert new keys and replace existing ones (what MERGE INTO does) without SQL:
      1. take the existing rows (only the touched months when the table is partitioned)
      2. drop the ones whose key arrives again (left_anti join)
      3. add the new rows and overwrite
    Running it twice with the same input leaves the same table (idempotent).
    """
    if not table_exists(spark, table):
        create_or_replace_table(new_df, table, location, partition_column)
        return

    existing = spark.table(table)
    new_keys = new_df.select(key).distinct()

    if partition_column:
        touched = new_df.select(F.date_trunc("month", partition_column).alias("_month")).distinct()
        existing = (existing.withColumn("_month", F.date_trunc("month", partition_column))
                            .join(touched, "_month", "left_semi")
                            .drop("_month"))

    result = existing.join(new_keys, key, "left_anti").unionByName(new_df)

    if partition_column:
        result.writeTo(table).overwritePartitions()   # replaces only the months in result
    else:
        result.writeTo(table).overwrite(F.lit(True))  # replaces the whole (small) table
```

Teoría del upsert. Las tablas Iceberg guardan archivos Parquet inmutables; “actualizar” significa escribir archivos nuevos y un snapshot nuevo. Con un left_anti join se descartan las filas existentes cuya clave llega de nuevo, se agregan las nuevas versiones y se sobrescribe. Si la tabla está particionada por mes, solo se leen y reescriben los meses tocados (`overwritePartitions`); si no lo está, es una tabla pequeña y se reemplaza completa. Ejecutarlo dos veces con el mismo lote deja la misma tabla (idempotente). **Costo:** order_items no está particionada, así que en el lote 2 full se reescribe completa; es el motivo de vigilar la compuerta de 3,0 USD.

helpers/keys.py — claves sustitutas y fila Unknown

```
"""Surrogate keys and the 'Unknown' row of each dimension."""
from pyspark.sql import functions as F


def surrogate_key(*columns):
    """Deterministic bigint key: same natural key -> same surrogate key on every run."""
    return F.abs(F.xxhash64(*columns))


def add_unknown_row(df, key_column, labels):
    """Append the row with key -1. labels: {column: text} for the descriptive columns."""
    values = []
    for field in df.schema.fields:
        if field.name == key_column:
            values.append(-1)
        else:
            values.append(labels.get(field.name))
    unknown = df.sparkSession.createDataFrame([tuple(values)], df.schema)
    return df.unionByName(unknown)
```

xxhash64 produce siempre el mismo número para la misma clave natural, así que no hace falta una tabla de secuencias y repetir el job no cambia las claves. add_unknown_row agrega la fila -1 que recibirán los hechos sin dimensión.

6.4 Job 1: bronze → silver (jobs/bronze_to_silver.py)

```
"""Job 1: bronze (raw_* tables) -> silver (Iceberg). Only the Spark DataFrame API is used."""
import sys

from awsglue.utils import getResolvedOptions
from pyspark.sql import functions as F

from helpers.cleaning import deduplicate, is_valid_email, lower_text, title_text, clean_text
from helpers.config import BRONZE_DB, SILVER_DB, get_bucket, table_name
from helpers.iceberg_io import read_bronze, upsert
from helpers.quality import quality_gate
from helpers.spark_session import create_session


# ---------- one function per entity: type + normalize, and the quality rules ----------

def prepare_customers(raw):
```

```
    df = raw.select(
        F.col("customer_id").cast("long").alias("customer_id"),
        title_text(F.col("first_name")).alias("first_name"),
        title_text(F.col("last_name")).alias("last_name"),
        lower_text(F.col("email")).alias("email"),
        F.to_date(F.col("birth_date")).alias("birth_date"),
        F.to_timestamp(F.col("updated_at")).alias("updated_at"),
    )
    rules = [
        ("missing_customer_id", F.col("customer_id").isNull()),
        ("invalid_email", ~is_valid_email(F.col("email"))),
    ]
    return df, rules, "updated_at"


def prepare_products(raw):
    df = raw.select(
        F.col("product_id").cast("long").alias("product_id"),
        clean_text(F.col("product_name")).alias("product_name"),
        title_text(F.col("category")).alias("category"),
        F.to_timestamp(F.col("updated_at")).alias("updated_at"),
    )
    rules = [
        ("missing_product_id", F.col("product_id").isNull()),
        ("missing_category", F.col("category").isNull()),
    ]
    return df, rules, "updated_at"


def prepare_stores(raw):
    df = raw.select(
        F.col("store_id").cast("long").alias("store_id"),
        clean_text(F.col("store_name")).alias("store_name"),
        title_text(F.col("city")).alias("city"),
        title_text(F.col("department")).alias("department"),
        F.to_timestamp(F.col("updated_at")).alias("updated_at"),
    )
    rules = [
        ("missing_store_id", F.col("store_id").isNull()),
        ("missing_location", F.col("city").isNull() | F.col("department").isNull()),
    ]
    return df, rules, "updated_at"


def prepare_orders(raw):
    df = raw.select(
        F.col("order_id").cast("long").alias("order_id"),
        F.col("customer_id").cast("long").alias("customer_id"),
        F.col("store_id").cast("long").alias("store_id"),
        F.to_date(F.col("order_date")).alias("order_date"),
        lower_text(F.col("status")).alias("status"),
        lower_text(F.col("payment_method")).alias("payment_method"),
        F.to_timestamp(F.col("updated_at")).alias("updated_at"),
    )
    rules = [
        ("missing_order_id", F.col("order_id").isNull()),
        ("missing_customer_id", F.col("customer_id").isNull()),
        ("missing_store_id", F.col("store_id").isNull()),
        ("invalid_order_date", F.col("order_date").isNull()),
        ("future_order_date", F.col("order_date") > F.current_date()),
        ("missing_status", F.col("status").isNull()),
    ]
    return df, rules, "updated_at"


def prepare_order_items(raw):
    df = raw.select(
        F.col("order_item_id").cast("long").alias("order_item_id"),
        F.col("order_id").cast("long").alias("order_id"),
        F.col("product_id").cast("long").alias("product_id"),
        F.col("quantity").cast("int").alias("quantity"),
        F.col("unit_price").cast("decimal(12,2)").alias("unit_price"),
        F.coalesce(F.col("discount_pct").cast("double"), F.lit(0.0)).alias("discount_pct"),
    )
    rules = [
        ("missing_order_item_id", F.col("order_item_id").isNull()),
        ("missing_order_id", F.col("order_id").isNull()),
        ("missing_product_id", F.col("product_id").isNull()),
        ("invalid_quantity", F.col("quantity").isNull() | (F.col("quantity") <= 0)),
        ("invalid_unit_price", F.col("unit_price").isNull() | (F.col("unit_price") < 0)),
        ("invalid_discount", (F.col("discount_pct") < 0) | (F.col("discount_pct") > 100)),
    ]
    return df, rules, None   # order items have no updated_at
```

```
# entity -> (bronze table, function, natural key, partition column)
ENTITIES = {
    "customers": ("raw_customers", prepare_customers, "customer_id", None),
    "products": ("raw_products", prepare_products, "product_id", None),
    "stores": ("raw_stores", prepare_stores, "store_id", None),
    "orders": ("raw_orders", prepare_orders, "order_id", "order_date"),
    "order_items": ("raw_order_items", prepare_order_items, "order_item_id", None),
}


def main():
    ingest_date = getResolvedOptions(sys.argv, ["ingest_date"])["ingest_date"]
    bucket = get_bucket()
    spark, glue_context = create_session(bucket)

    for entity, (bronze_table, prepare, key, partition_column) in ENTITIES.items():
        raw = read_bronze(glue_context, BRONZE_DB, bronze_table, ingest_date)
        typed, rules, order_column = prepare(raw)

        # validate first, so rows without key do not collapse into one in the dedupe
        rejects_path = f"s3://{bucket}/silver/_rejects/{entity}/ingest_date={ingest_date}/"
        valid = quality_gate(typed, entity, rules, rejects_path)

        unique = deduplicate(valid, key, order_column) if order_column \
            else valid.dropDuplicates([key])

        final = (unique
                 .withColumn("_ingest_date", F.lit(ingest_date).cast("date"))
                 .withColumn("_loaded_at", F.current_timestamp()))

        upsert(spark, final, table_name(SILVER_DB, entity), key,
               location=f"s3://{bucket}/silver/{entity}",
               partition_column=partition_column)
        print(f"[SILVER] entity={entity} written for ingest_date={ingest_date}")


main()
```

Cada `prepare_*` devuelve el DataFrame tipificado, la lista de reglas y la columna que decide cuál duplicado es el más reciente. Un diccionario `ENTITIES` describe las 5 entidades y un solo bucle las procesa: agregar una entidad es agregar una entrada. Se agregan dos columnas de auditoría: `_ingest_date` (qué lote la tocó por última vez, la usa gold) y `_loaded_at`.

6.5 Job 2: silver → gold (jobs/silver_to_gold.py)

```
"""Job 2: silver -> gold (star schema in Iceberg). Only the Spark DataFrame API is used."""
import sys
from datetime import date

from awsglue.utils import getResolvedOptions
from pyspark.sql import functions as F

from helpers.config import DATE_END, DATE_START, GOLD_DB, SILVER_DB, get_bucket, table_name
from helpers.iceberg_io import create_or_replace_table, upsert
from helpers.keys import add_unknown_row, surrogate_key
from helpers.spark_session import create_session


# ---------- dimensions (no PII: names and emails stay in silver) ----------

def build_dim_customer(customers):
    age = F.floor(F.months_between(F.current_date(), F.col("birth_date")) / 12)
    age_group = (F.when(age.isNull(), "Unknown")
                  .when(age < 18, "Under 18")
                  .when(age <= 25, "18-25")
                  .when(age <= 35, "26-35")
                  .when(age <= 45, "36-45")
                  .when(age <= 60, "46-60")
                  .otherwise("61+"))
    df = customers.select(
        surrogate_key(F.col("customer_id")).alias("customer_key"),
        F.col("customer_id"),
        age_group.alias("age_group"),
    )
    return add_unknown_row(df, "customer_key", {"age_group": "Unknown"})


def build_dim_product(products):
    df = products.select(
        surrogate_key(F.col("product_id")).alias("product_key"),
        F.col("product_id"), F.col("product_name"), F.col("category"),
```

```
    )
    return add_unknown_row(df, "product_key",
                           {"product_name": "Unknown", "category": "Unknown"})


def build_dim_store(stores):
    df = stores.select(
        surrogate_key(F.col("store_id")).alias("store_key"),
        F.col("store_id"), F.col("store_name"),
    )
    return add_unknown_row(df, "store_key", {"store_name": "Unknown"})


def build_dim_geography(stores):
    df = (stores.select("department", "city").distinct()
                .select(surrogate_key(F.col("department"), F.col("city")).alias("geography_key"),
                        F.col("department"), F.col("city")))
    return add_unknown_row(df, "geography_key", {"department": "Unknown", "city": "Unknown"})


def build_dim_date(spark):
    days = (date.fromisoformat(DATE_END) - date.fromisoformat(DATE_START)).days + 1
    df = (spark.range(days)
               .select(F.date_add(F.lit(DATE_START).cast("date"),
                                  F.col("id").cast("int")).alias("full_date")))
    df = df.select(
        F.date_format("full_date", "yyyyMMdd").cast("int").alias("date_key"),
        F.col("full_date"),
        F.year("full_date").alias("year"),
        F.quarter("full_date").alias("quarter"),
        F.month("full_date").alias("month"),
        F.date_format("full_date", "MMMM").alias("month_name"),
        F.dayofmonth("full_date").alias("day_of_month"),
        F.dayofweek("full_date").alias("day_of_week"),
        F.dayofweek("full_date").isin(1, 7).alias("is_weekend"),
    )
    return add_unknown_row(df, "date_key", {"month_name": "Unknown"})


# ---------- fact table ----------

def build_fact_sales(orders, items, stores, dims, ingest_date):
    # only the orders touched by this batch (new or updated)
    touched = (orders.filter(F.col("_ingest_date") == F.lit(ingest_date).cast("date"))
                     .select("order_id", "customer_id", "store_id", "order_date",
                             F.col("status").alias("order_status"), "payment_method"))

    store_geography = stores.select(
        "store_id",
        surrogate_key(F.col("department"), F.col("city")).alias("geography_key"))

    lines = (items.join(touched, "order_id", "inner")
                  .join(F.broadcast(dims["customer"].select("customer_id", "customer_key")),
                        "customer_id", "left")
                  .join(F.broadcast(dims["product"].select("product_id", "product_key")),
                        "product_id", "left")
                  .join(F.broadcast(dims["store"].select("store_id", "store_key")),
                        "store_id", "left")
                  .join(F.broadcast(store_geography), "store_id", "left")
                  .na.fill(-1, ["customer_key", "product_key", "store_key", "geography_key"]))

    gross = (F.col("quantity") * F.col("unit_price")).cast("decimal(14,2)")
    discount = F.round(F.col("gross_amount") * F.col("discount_pct") / 100, 2)

    return (lines
            .withColumn("gross_amount", gross)
            .withColumn("discount_amount", discount.cast("decimal(14,2)"))
            .withColumn("net_amount",
                        (F.col("gross_amount") - F.col("discount_amount")).cast("decimal(14,2)"))
            .withColumn("date_key", F.date_format("order_date", "yyyyMMdd").cast("int"))
            .withColumn("_loaded_at", F.current_timestamp())
            .select("order_item_id", "order_id", "date_key",
                    "customer_key", "product_key", "store_key", "geography_key",
                    "order_date", "quantity", "discount_pct", "unit_price",
                    "gross_amount", "discount_amount", "net_amount",
                    "order_status", "payment_method", "_loaded_at"))


# ---------- data mart ----------

def build_mart(fact, dims):
    return (fact.filter(~F.col("order_status").isin("cancelled", "returned"))
                .join(F.broadcast(dims["date"].select("date_key", "year", "quarter")), "date_key")
                .join(F.broadcast(dims["customer"].select("customer_key", "age_group")),
```

```
                  "customer_key")
            .join(F.broadcast(dims["geography"].select("geography_key", "department")),
                  "geography_key")
            .groupBy("year", "quarter", "age_group", "department")
            .agg(F.countDistinct("order_id").alias("orders"),
                 F.sum("quantity").alias("units"),
                 F.sum("net_amount").alias("net_sales")))


def main():
    ingest_date = getResolvedOptions(sys.argv, ["ingest_date"])["ingest_date"]
    bucket = get_bucket()
    spark, _ = create_session(bucket)

    silver = lambda name: spark.table(table_name(SILVER_DB, name))
    gold_location = lambda name: f"s3://{bucket}/gold/{name}"

    customers, products, stores = silver("customers"), silver("products"), silver("stores")
    orders, items = silver("orders"), silver("order_items")

    # dimensions: rebuilt from silver on every run (SCD type 1)
    dims = {
        "customer": build_dim_customer(customers),
        "product": build_dim_product(products),
        "store": build_dim_store(stores),
        "geography": build_dim_geography(stores),
        "date": build_dim_date(spark),
    }
    for name, df in dims.items():
        create_or_replace_table(df, table_name(GOLD_DB, f"dim_{name}"), gold_location(f"dim_{name}"))
        print(f"[GOLD] dim_{name} written")

    # fact: upsert of the touched orders, partitioned by month
    fact_new = build_fact_sales(orders, items, stores, dims, ingest_date)
    upsert(spark, fact_new, table_name(GOLD_DB, "fact_sales"), "order_item_id",
           location=gold_location("fact_sales"), partition_column="order_date")
    print("[GOLD] fact_sales upserted")

    # mart: recomputed from the whole fact table
    fact_all = spark.table(table_name(GOLD_DB, "fact_sales"))
    create_or_replace_table(build_mart(fact_all, dims),
                            table_name(GOLD_DB, "agg_sales_dept_age_quarter"),
                            gold_location("agg_sales_dept_age_quarter"))
    print("[GOLD] agg_sales_dept_age_quarter written")


main()
```

Teoría del modelo estrella. La tabla de hechos guarda medidas (cantidad, importes) y claves hacia las dimensiones, que describen el contexto (cliente, producto, tienda, lugar, fecha). Las dimensiones se reconstruyen completas en cada corrida (SCD tipo 1: el valor nuevo pisa al anterior), por eso también reflejan un borrado en silver. La tabla de hechos usa upsert solo con los pedidos tocados por el lote (`_ingest_date` = fecha del lote), incluidos los pedidos actualizados a returned. Un join left más `na.fill(-1)` asigna la clave -1 cuando falta la dimensión. dim_geography se deriva de la ubicación de la tienda. El mart se recalcula desde toda la tabla de hechos.

6.6 Probar y orquestar

```
python -m py_compile jobs/*.py helpers/*.py
cd terraform && terraform plan -out=tfplan && terraform apply tfplan     # uploads the real scripts

# one job alone
aws glue start-job-run --job-name lakehouse-bronze-to-silver --arguments '{"--ingest_date":"2026-11-07"}'
aws glue get-job-run --job-name lakehouse-bronze-to-silver --run-id <JobRunId> --query JobRun.JobRunState

# full pipeline
SM=$(terraform output -raw state_machine_arn)
aws stepfunctions start-execution --state-machine-arn $SM --input '{"ingest_date":"2026-11-07"}'
```

- Cuando editen un script o un helper, repitan terraform plan -out=tfplan y apply: es lo que sube la versión nueva al bucket (si aplicaron el Anexo A, antes regeneren helpers.zip).
- Logs en CloudWatch → Log groups → /aws-glue/jobs/output (líneas [QUALITY], [SILVER] y [GOLD]).
- Si Glue falla con “extensions not found”, agregar a default_arguments el parámetro "--conf" con spark.sql.extensions=org.apache.iceberg.spark.extensions.IcebergSparkSessionExtensions.
- Si Iceberg falla con Glue 5.0, probar glue_version = "4.0".

- Si falla `ModuleNotFoundError: No module named 'helpers'`, Glue no recibió la carpeta helpers: revisar el Anexo A.

Sin probar en Glue. El código se revisó y compila (py_compile), pero no se ejecutó contra Spark/Iceberg real. Hagan la primera corrida con el perfil mini (2 workers) y revisen los logs antes de ir al full.

Evidencias 12 a 17: run de silver y de gold en Succeeded, líneas [QUALITY], tablas de silver (5) y gold (7), y el grafo verde de Step Functions.

7. Lote 2 y evidencia de upsert (martes, Avance 2)

```
cd data
python generate_data.py --profile mini --batch 2 --ingest-date 2026-11-09 --batch1-date 2026-11-07
BUCKET=$(cd ../terraform && terraform output -raw bucket_name)
aws s3 sync out/landing s3://$BUCKET/landing/

-- BEFORE running the pipeline, in Athena
MSCK REPAIR TABLE lakehouse_bronze.raw_customers;   -- same for raw_products, raw_stores,
MSCK REPAIR TABLE lakehouse_bronze.raw_orders;      -- raw_order_items and raw_web_events
SELECT count(*) FROM lakehouse_silver.orders;                           -- ~50,000
SELECT count(*) FROM lakehouse_silver.orders WHERE status = 'returned';  -- write it down
SELECT current_timestamp;                                               -- write it down (UTC)
```

Correr el pipeline con {"ingest_date":"2026-11-09"}. Esperado: silver.orders ≈ 54.000 (+4.000 insertados) y returned sube en 1.000 (actualizados). Esa es la evidencia del upsert.

8. SQL de negocio y capacidades lakehouse (miércoles)

Estas consultas se ejecutan en Athena (no en Spark).

8.1 Preguntas de negocio (sql/04_questions.sql)

Cada una va al README con consulta, resultado e interpretación (2 a 3 líneas) y con el tiempo y Data scanned que muestra Athena.

```
-- Q1: top 3 departments by net sales in each quarter
WITH v AS (
  SELECT d.year, d.quarter, g.department, SUM(f.net_amount) AS sales
  FROM lakehouse_gold.fact_sales f
  JOIN lakehouse_gold.dim_date d ON f.date_key = d.date_key
  JOIN lakehouse_gold.dim_geography g ON f.geography_key = g.geography_key
  WHERE f.order_status NOT IN ('cancelled', 'returned')
  GROUP BY 1, 2, 3)
SELECT * FROM (SELECT *, RANK() OVER (PARTITION BY year, quarter ORDER BY sales DESC) AS rk FROM v)
WHERE rk <= 3 ORDER BY year, quarter, rk;

-- Q2: sales and average ticket by age group
SELECT c.age_group, COUNT(DISTINCT f.order_id) AS orders, SUM(f.net_amount) AS sales,
       ROUND(SUM(f.net_amount) / COUNT(DISTINCT f.order_id), 0) AS average_ticket
FROM lakehouse_gold.fact_sales f
JOIN lakehouse_gold.dim_customer c ON f.customer_key = c.customer_key
WHERE f.order_status NOT IN ('cancelled', 'returned')
GROUP BY 1 ORDER BY sales DESC;

-- Q3 (manager): leading department by age group and quarter
WITH v AS (
  SELECT d.year, d.quarter, c.age_group, g.department, SUM(f.net_amount) AS sales
  FROM lakehouse_gold.fact_sales f
  JOIN lakehouse_gold.dim_date d ON f.date_key = d.date_key
  JOIN lakehouse_gold.dim_customer c ON f.customer_key = c.customer_key
  JOIN lakehouse_gold.dim_geography g ON f.geography_key = g.geography_key
  WHERE f.order_status NOT IN ('cancelled', 'returned')
  GROUP BY 1, 2, 3, 4)
SELECT year, quarter, age_group, department, sales FROM (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY year, quarter, age_group ORDER BY sales DESC) AS rn
  FROM v)
WHERE rn = 1 ORDER BY year, quarter, age_group;

-- Q3 on the data mart (same answer, no joins)
SELECT year, quarter, age_group, department, net_sales FROM (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY year, quarter, age_group ORDER BY net_sales DESC) AS rn
  FROM lakehouse_gold.agg_sales_dept_age_quarter)
WHERE rn = 1 ORDER BY year, quarter, age_group;
```

Argumento: en el OLTP la pregunta exigía 8 o más joins; en la estrella son 3 o 4, y con el mart son 0. Bonus (Q4): categorías con más devoluciones.

8.2 Time travel, UPDATE, DELETE y mantenimiento (sql/05_lakehouse.sql)

```
SELECT snapshot_id, committed_at, operation FROM "lakehouse_silver"."orders$snapshots" ORDER BY committed_at;
SELECT count(*) FROM lakehouse_silver.orders FOR VERSION AS OF <snapshot_before_the_upsert>;
SELECT count(*) FROM lakehouse_silver.orders FOR TIMESTAMP AS OF TIMESTAMP '2026-11-09 15:00:00 UTC';
SELECT count(*) FROM lakehouse_silver.orders;                  -- current: compare

UPDATE lakehouse_silver.orders SET status = 'returned', updated_at = current_timestamp
WHERE order_id = 12345;

-- Right to be forgotten (GDPR), customer 1234
SELECT customer_key FROM lakehouse_gold.dim_customer WHERE customer_id = 1234;   -- write it down
DELETE FROM lakehouse_gold.fact_sales WHERE customer_key = <noted_value>;
DELETE FROM lakehouse_gold.dim_customer WHERE customer_id = 1234;
DELETE FROM lakehouse_silver.order_items
WHERE order_id IN (SELECT order_id FROM lakehouse_silver.orders WHERE customer_id = 1234);
DELETE FROM lakehouse_silver.orders WHERE customer_id = 1234;
DELETE FROM lakehouse_silver.customers WHERE customer_id = 1234;
SELECT count(*) FROM lakehouse_silver.customers WHERE customer_id = 1234;       -- 0

OPTIMIZE lakehouse_gold.fact_sales REWRITE DATA USING BIN_PACK;
VACUUM lakehouse_silver.customers;       -- physical purge of expired snapshots
```

Si Athena rechaza la subconsulta del DELETE sobre order_items, sacar antes los order_id con un SELECT y pegarlos en el IN (...). El DELETE es lógico hasta que corre VACUUM; el mart se recalcula en la siguiente corrida.

8.3 Volumen e integridad (sql/06_volume.sql)

```
-- Row count per layer (complete with the tables of each layer)
SELECT 'gold' layer, 'fact_sales' table_name, count(*) row_count FROM lakehouse_gold.fact_sales;
SELECT count(*) AS files, round(sum(file_size_in_bytes)/1e6, 1) AS mb FROM "lakehouse_gold"."fact_sales$files";
SELECT * FROM "lakehouse_gold"."fact_sales$partitions" LIMIT 30;

-- Referential integrity: all must return 0
SELECT count(*) FROM lakehouse_gold.fact_sales f
LEFT JOIN lakehouse_gold.dim_product p ON f.product_key = p.product_key
WHERE p.product_key IS NULL;
```

El archivo sql/06_volume.sql completo (conteo de las tablas de cada capa y las tres comprobaciones de integridad: producto, cliente y tienda) está en la sección 8.3 del documento editable.

9. Corrida full de 10M+ (miércoles en la noche y jueves)

9.1 Miércoles en la noche (gratis, en su computador)

```
python generate_data.py --profile full --batch 1 --ingest-date 2026-11-11 --out out_full
```

Verificar du -sh out_full/landing (unos 250 a 350 MB) y unos 42 archivos de ítems. El proceso usa ~2 a 3 GB de RAM.

9.2 Jueves: lote 1 full (quien ejecuta apply)

- Compuerta de costos: comando de gasto real ≤ 1,5 USD.
- En terraform.tfvars: glue_workers = 8 y glue_timeout_min = 25. Ejecutar terraform destroy y luego terraform apply: deja el ambiente limpio y demuestra reproducibilidad.
- aws s3 sync ../data/out_full/landing s3://$BUCKET/landing/ y arrancar los 2 crawlers.
- En Athena: SELECT count(*) FROM lakehouse_bronze.raw_order_items; (≈ 10,6M).
- Correr el pipeline una sola vez con {"ingest_date":"2026-11-11"}.
- Vigilar en Glue → ETL jobs → Runs: si un job pasa de 25 min, el timeout lo corta. No lanzar otro sin revisar el log.
- Correr sql/06_volume.sql y capturar fact_sales (≥ 10M), el total entre capas, Data scanned y el tiempo.

9.3 Jueves en la tarde: lote 2 full (solo si el acumulado es ≤ 3,0 USD)

Generar el lote 2 full, subirlo, correr MSCK REPAIR TABLE en las 6 tablas bronze, anotar cifras previas y ejecutar el pipeline con {"ingest_date":"2026-11-12"}. Luego correr sql/04, 05 y 06 sobre los datos full: esa es la sustentación. No ejecutar terraform destroy hasta el viernes. Si el acumulado supera 3,0 USD, presentar el upsert con las evidencias del mini.

10. README, sustentación y cierre

Estructura del README: descripción del problema, diagrama, decisiones (Iceberg vs Parquet plano, surrogate keys, 1 bucket con prefijos, SCD1, Spark nativo con upsert en lugar de MERGE INTO, por qué silver tiene 5 tablas y gold 7), arquitectura medallón, estructura jobs/ y helpers/, cómo desplegar y correr el pipeline, calidad de datos, modelo estrella con diccionario, las 3+ preguntas con interpretación, upsert/UPDATE/DELETE/time travel con capturas, tabla de volumen, costos reales vs presupuesto, seguridad y terraform destroy. Incluir el bootstrap manual de la cuenta (sección 3).

Respuesta a la crítica del número de tablas: silver conserva una tabla por entidad transaccional (5), que son las que necesita gold. Los logs web_events se quedan en bronze porque ningún requisito de negocio los consume. Gold tiene 7: fact_sales, 5 dimensiones y un mart.

Respuesta a "¿por qué no MERGE INTO?": porque Iceberg no lo expone en la API de DataFrames de PySpark; se implementó el mismo comportamiento con un anti-join y escritura dinámica por partición, con la ventaja de que cada paso se puede explicar y probar por separado.

Guion de 10 minutos

Min	Contenido
1	Problema y arquitectura
2	Terraform: un apply, tags, roles mínimos, Budgets
2	Pipeline: Step Functions, helpers y calidad con rechazos
2	Volumen 10M+, estrella y preguntas
2	Upsert, DELETE y time travel
1	Costos reales y terraform destroy en vivo

Ambos deben saber responder todo.

Cierre (viernes, al final de la demo)

```
terraform destroy
aws s3 ls | grep lakehouse                              # must print nothing
aws glue get-databases --query "DatabaseList[].Name"    # no lakehouse_* databases
```

11. Mapa de evidencias

Todas van en docs/evidence/ con nombre numerado y se referencian en el README. Avance 1: filas 1 a 8, diagrama y ficha. Avance 2: filas 9 a 18.

N.º	Fase	Qué capturar	Archivo	Rúbrica
1	Terraform	Apply complete! Resources: 31 added	01-apply.png	IaC 20 %
2	Terraform	S3: bucket con sus prefijos	02-s3.png	IaC
3	Terraform	Budgets: 4 alertas	03-budgets.png	Seguridad 5 %
4	Terraform	Athena: workgroup lakehouse-wg, engine v3	04-athena.png	IaC
5	Terraform	Glue: 3 bases de datos	05-glue-dbs.png	IaC
6	Terraform	Glue: los 2 jobs	06-glue-jobs.png	IaC
7	Terraform	Step Functions: lakehouse-pipeline	07-sfn.png	IaC
8	Terraform	IAM: 3 roles y usuario con MFA	08-iam.png	Seguridad
9	Landing	S3 landing/ con ingest_date=...	09-landing.png	Landing 15 %
10	Bronze	Crawlers en Succeeded	10-crawlers.png	Landing
11	Bronze	Tablas raw_* y conteo en Athena	11-bronze.png	Landing
12	Silver	Run bronze-to-silver en Succeeded (duración, DPU-horas)	12-job-silver.png	Medallón 20 %
13	Silver	CloudWatch: líneas [QUALITY]	13-quality-logs.png	Medallón
14	Silver	silver/_rejects/ y 5 tablas de silver	14-silver.png	Medallón
15	Gold	Run silver-to-gold en Succeeded	15-job-gold.png	Medallón
16	Gold	7 tablas de gold en Glue	16-gold.png	Modelo 15 %
17	Pipeline	Step Functions: grafo en verde	17-sfn-ok.png	IaC / Medallón
18	Lote 2	Conteos antes/después (+4.000) y returned (+1.000)	18-merge.png	Lakehouse 5 %
19	SQL	Q1, Q2 y Q3 con resultado, tiempo y Data scanned	19-q1.png a 21-q3.png	Preguntas 10 %
20	SQL	$snapshots, time travel y conteo actual	22-timetravel.png	Lakehouse
21	SQL	UPDATE y DELETE con verificación en 0	23-update-delete.png	Lakehouse
22	Full	fact_sales ≥ 10M y total entre capas	24-volumen.png	Medallón / Modelo
23	Full	Duración y DPU-horas de la corrida full	25-glue-full.png	Costos
24	Full	Integridad referencial en 0	26-integridad.png	Modelo
25	Costos	Cost Explorer y gasto real de Glue	27-costos.png	Costos
26	Cierre	terraform destroy y verificación	28-destroy.png	Costos

12. Cronograma

Día	Meta
Sáb 7	Cuenta, lakehouse-admin, CLI, repo y terraform apply con placeholders. Generador mini, landing, crawlers y bronze. El Avance 1 (06/11) se presenta con el diagrama y lo desplegado.
Dom 8	Job 1 con Iceberg particionado.
Lun 9	Job 2 (dimensiones, hechos, mart) y pipeline en Step Functions.
Mar 10	Lote 2 mini con upsert y evidencias en GitHub (Avance 2).
Mié 11	Preguntas, UPDATE/DELETE y time travel en mini. De noche, generar el full local.
Jue 12	Compuerta de costos, destroy/apply, full lote 1, evidencias de 10M, lote 2 full si el presupuesto lo permite, README y ensayo.
Vie 13	Demo sobre los datos full y terraform destroy en vivo.

13. Errores típicos

Síntoma	Solución
Columnas col0, col1 en bronze	Revisar que el clasificador CSV esté asociado al crawler y volver a correrlo.
Job falla en writeTo / createOrReplace	Falta --datalake-formats iceberg o no sirve esa versión de Glue. Probar glue_version = "4.0".
ModuleNotFoundError: helpers	Glue no recibió la carpeta helpers/. Aplicar el Anexo A y verificar que helpers.zip tiene la carpeta helpers/ en su raíz.
AnalysisException: cannot resolve column ...	El nombre de columna de bronze no coincide con el esperado. Comparar con DESCRIBE y ajustar la función prepare_* (sección 5).
AccessDenied en un job	Falta una acción en glue_perms (rol de Glue). CloudWatch dice cuál; corregir en iam.tf y reaplicar.
push_down_predicate falla	ingest_date quedó como date. read_bronze ya hace el respaldo: filtra después de toDF().
ValueError: reject rate ... above 5 %	Revisar silver/_rejects/<entidad>/ y el motivo en _reject_reason; corregir la regla o el dato de origen.
Duplicados en silver tras el upsert	La clave no es única en el lote: la deduplicación debe correr antes del upsert (ya ocurre en bronze_to_silver).
Athena no ve particiones nuevas	MSCK REPAIR TABLE en cada tabla bronze.
Insufficient Lake Formation permission	Lake Formation → Data catalog settings: marcar Use only IAM access control.
ResourceNumberLimitExceeded	Cuota de DPUs baja en cuentas nuevas: pedir aumento en Service Quotas o bajar glue_workers.
Terraform: Budgets AccessDenied	Falta activar el acceso de IAM a Billing con el usuario root.
La consola muestra “no data”	Región equivocada: debe decir N. Virginia.
BucketAlreadyExists	Otra cuenta usa el nombre; cambiar project en variables.tf.

14. Convenciones de trabajo

- **Idioma del código:** todo en inglés (variables, funciones, columnas, comentarios, mensajes de log y de error). La documentación y el README van en español.
- **Estilo del código:** simple y fácil de explicar. Funciones cortas, una responsabilidad por archivo de helpers/, sin trucos: si algo no se puede explicar en una frase, se divide.
- **Spark:** solo API de DataFrames. Nada de spark.sql, expr ni selectExpr.
- **Commits (Conventional Commits):** tipo(alcance): descripción en minúsculas. Tipos: feat, fix, docs, chore, refactor. Ejemplo: feat(glue): add bronze to silver job with data quality gate. Un solo idioma en todo el historial.
- **Terminal:** Git Bash usa export AWS_PROFILE=lakehouse; PowerShell usa $env:AWS_PROFILE = "lakehouse". No mezclarlas.
- **Terraform:** fmt, validate y plan -out=tfplan antes de cada apply tfplan. Nunca versionar terraform.tfvars, tfplan, .terraform/ ni el estado.
- **HCL:** un bloque de una sola línea solo puede contener atributos; los bloques anidados van en varias líneas.
- **Credenciales:** el CSV de la access key y la carpeta ~/.aws/ nunca van dentro del repositorio.
- **Consola:** solo para mirar y capturar. Los cambios de infraestructura se hacen en los .tf.

Anexo A. Que Glue reciba helpers/ (cambio opcional en glue.tf)

Hoy glue.tf sube solo `../jobs/*.py`. Un job de Glue no ve las demás carpetas del repositorio, así que `from helpers.config import ...` fallaría. Glue admite el parámetro `--extra-py-files`, que apunta a un **.zip en S3** que se agrega al path de Python. Hay que hacer tres cosas. No se tocó ningún archivo de Terraform en esta versión: este anexo es solo la receta, y los nombres de recursos (aws_s3_bucket.lakehouse, aws_glue_job.*) hay que adaptarlos a los de su glue.tf.

1. Generar el zip (desde la raíz del repo, antes de cada plan)

```
# the zip must contain the folder "helpers/" at its root
zip -r terraform/helpers.zip helpers -x "*__pycache__*"
# PowerShell: Compress-Archive -Path helpers -DestinationPath terraform\helpers.zip -Force
```

2. Subir el zip junto con los scripts (glue.tf)

```
resource "aws_s3_object" "helpers_zip" {
  bucket = aws_s3_bucket.lakehouse.id               # adapt to your bucket resource name
  key    = "scripts/helpers.zip"
  source = "${path.module}/helpers.zip"
  etag   = filemd5("${path.module}/helpers.zip")
}
```

3. Pasar el zip a los dos jobs (default_arguments de cada aws_glue_job)

```
"--extra-py-files" = "s3://${aws_s3_bucket.lakehouse.id}/scripts/helpers.zip"
```

Con este cambio el plan mostrará 1 recurso más (32 to add en un ambiente limpio). Si prefieren no tocar glue.tf, la alternativa es copiar el contenido de helpers/ dentro de cada script de jobs/, pero se pierde la reutilización y la carpeta helpers/ deja de tener sentido.